

ExoHabitAI

Milestone 3 - Visualization & Deployment Documentation

Name: Satya Sri Dheeraj M
Date: 14 February 2026

1. Overview

This milestone documents the full deployment of ExoHabitAI - a machine learning-powered exoplanet habitability prediction system. The application is live and accessible via two separately deployed services: a Next.js frontend on Vercel and a Flask REST API backend on Render. The trained Logistic Regression model is bundled with the backend and served in real time.

Live Dashboard: <https://exohabitai-frontend.vercel.app/dashboard>

Backend API [new version]: <https://exohabitai-backend.onrender.com>

2. Deployment Architecture

The system uses a decoupled deployment architecture with the frontend and backend hosted on separate platforms. This separation allows independent scaling, deployment, and versioning of each service.

Component	Platform	Repository	Status
Frontend (Next.js)	Vercel	CrownDestro/ExoHabitAI-Frontend	Live
Backend (Flask API)	Render	CrownDestro/ExoHabitAI-Backend	Live

Frontend requests are proxied to the backend via Next.js rewrites configured in `next.config.mjs`. The `NEXT_PUBLIC_API_URL` environment variable on Vercel points to the live Render URL, enabling seamless cross-origin communication without exposing the backend URL in client-side code.

3. Frontend Deployment - Vercel

a. Platform

Vercel was selected for its native Next.js support, automatic CI/CD on every GitHub push, global CDN distribution, and free tier availability. Build times average under 60 seconds with zero configuration required beyond connecting the repository.

b. Configuration

The following environment variable was set in the Vercel project dashboard before deployment:

```
NEXT_PUBLIC_API_URL=https://exohabitai-backend.onrender.com
```

The Next.js build command (`npm run build`) compiles TypeScript, bundles Three.js components with dynamic imports (`ssr:false`), and produces an optimized static and server-rendered output. Three.js components are excluded from server-side rendering to prevent WebGL hydration errors.

c. Deployment Steps

- 1. Connected CrownDestro/ExoHabitAI-Frontend repository to Vercel via GitHub OAuth.
- 2. Set NEXT_PUBLIC_API_URL environment variable in project settings.
- 3. Triggered initial deployment — Vercel ran `npm install` and `npm run build` automatically.
- 4. Build completed successfully. Application live at `vercel.app` domain.
- 5. Subsequent pushes to the main branch trigger automatic redeployment.

4. Backend Deployment — Render

a. Platform

Render was selected for its native Python/Flask support, automatic WSGI server configuration via Gunicorn, free tier web service hosting, and direct GitHub integration. The free tier provides 750 instance hours per month — sufficient for demonstration and academic use.

b. Configuration

Setting	Value
Build Command	<code>pip install -r requirements.txt</code>
Start Command	<code>gunicorn app:app</code>
Python Version	3.12

c. Requirements

The following key dependencies are specified in `requirements.txt` for the Render deployment environment:

Package	Version	Purpose
Flask	3.0.0	Web framework and routing
flask-cors	4.0.0	Cross-origin request handling
scikit-learn	1.8.0	Model loading and inference
pandas	2.0+	Data processing and CSV handling

numpy	1.24+	Numerical computations
gunicorn	21.0+	Production WSGI server
xgboost	2.0+	Alternative model support

5. Live API Endpoints

All endpoints are live and accessible at the Render deployment URL:

Method	Endpoint	Description
GET	/health	Model and server health check
GET	/rank?top=20&threshold=0.0	Pre-computed habitability rankings
POST	/predict	Single planet habitability prediction
POST	/batch_predict	Batch predictions (max 100 planets)
GET	/examples	Example input payloads
GET	/	API documentation

6. Model in Production

a. Model Details

Property	Value
Algorithm	Logistic Regression with Platt Scaling
Features	15 (6 numerical + 9 one-hot encoded categorical)
Training Set	3,231 unique exoplanets
Test Set	796 unique exoplanets
CV Strategy	GroupKFold (5-fold, star system-aware)
CV Recall	87.0% $\pm$ 16.6%
Test ROC-AUC	0.9852

Average Precision	0.4673
Recall@10	42.9% (3/7 habitable in top 10)
Recall@50	85.7% (6/7 habitable in top 50)
Class Imbalance	148:1 (non-habitable to habitable)
Serialization	pickle protocol 4

b. Inference Pipeline

On each POST /predict request, the following pipeline executes: JSON parsing and content-type validation → input parameter validation against physical constraints → feature vector construction with one-hot encoding → `model.predict_proba()` returning calibrated probability → response formatting with category, confidence, and observation recommendation → JSON serialization.

c. Fine-Tuning Decisions

The production model reflects several deliberate fine-tuning choices made during the training phase:

Class Weight Balancing

`class_weight='balanced'` was applied during Logistic Regression training to compensate for the 148:1 class imbalance. This prevents the model from simply predicting all planets as non-habitable (which would achieve 99.3% accuracy but 0% recall on habitable planets).

Probability Calibration

`CalibratedClassifierCV` with Platt scaling (sigmoid method) was applied after initial training. This ensures predicted probabilities reflect genuine uncertainty, making the ranking system meaningful. Uncalibrated probabilities from Logistic Regression can be overconfident near the decision boundary.

Decision Threshold

The classification threshold was kept at 0.5 (default) for binary predictions. However, the ranking system uses raw probabilities rather than binary predictions, enabling nuanced prioritization of candidates. The dashboard displays all planets ranked by probability regardless of the binary threshold.

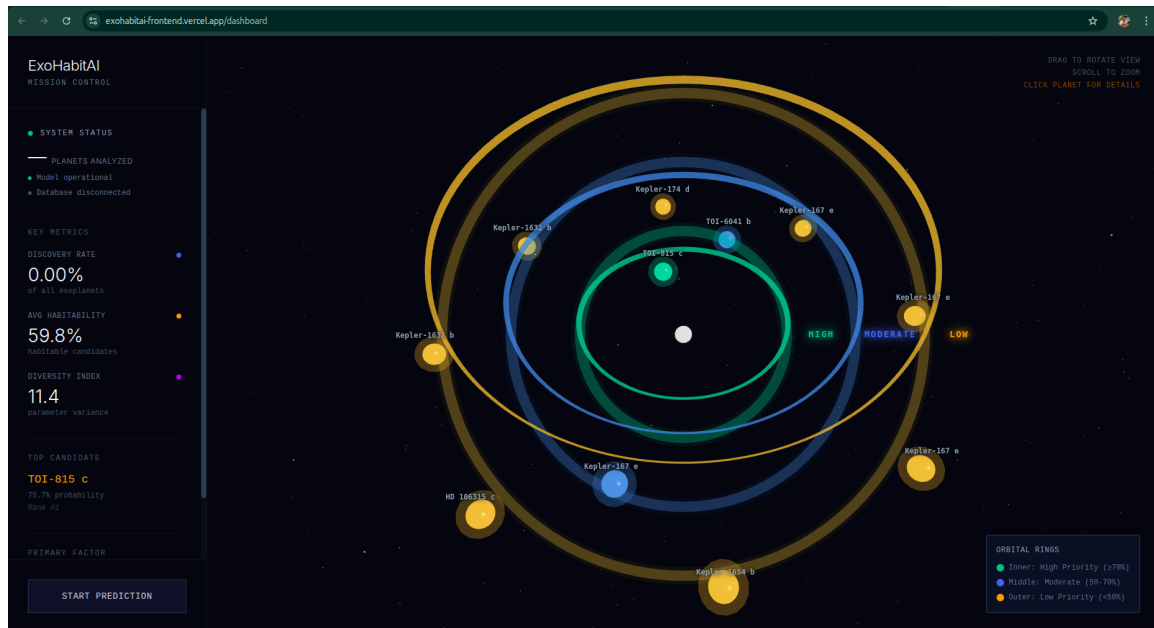
Feature Leakage Prevention

11 features were excluded from training to prevent data leakage: 5 features directly used in creating habitability labels (orbital flux, equilibrium temperature, planet radius, orbital eccentricity, stellar effective temperature) and 6 proxy features that enable reconstruction of those values. This forces the model to learn indirect correlations and ensures predictions on new data are scientifically meaningful.

7. Screenshots

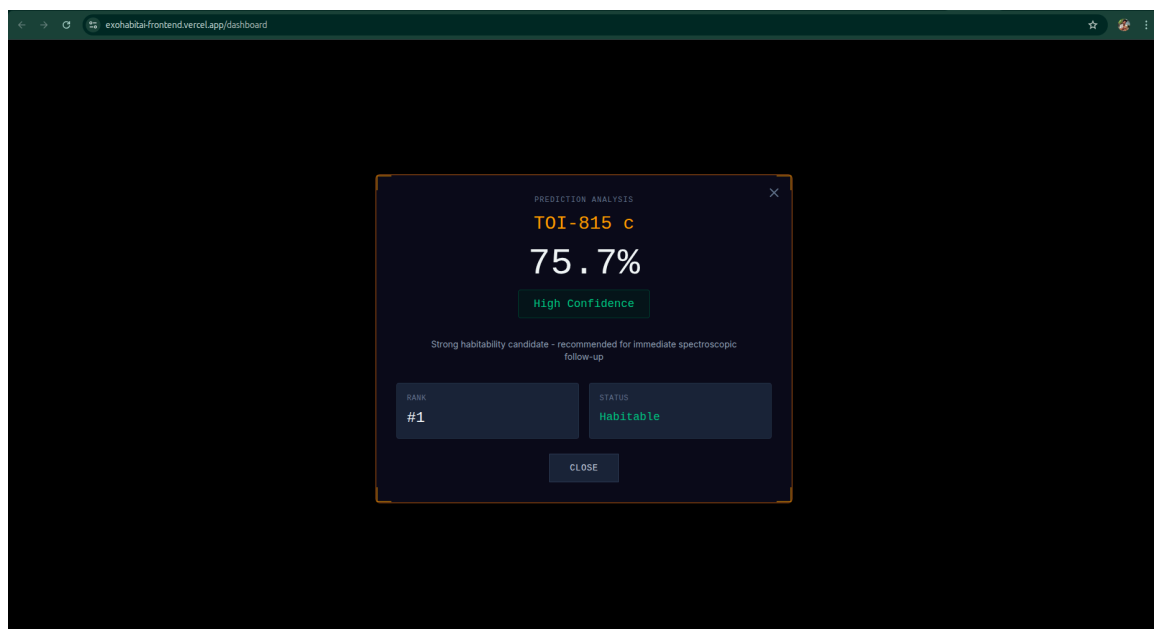
[Screenshot 1: Live Dashboard — <https://exohabitai-frontend.vercel.app/dashboard>]

Shows the Mission Control interface with three orbital rings (green for high confidence, blue for moderate, amber for low), glowing planets orbiting each ring, left statistics sidebar, and orbital legend.



[Screenshot 2: Planet Click — Prediction Modal]

Shows the prediction details modal with planet name, habitability percentage, confidence category badge, rank and status cards, and close button.



[Screenshot 3: Prediction Form — /predict route]
Shows the multi-parameter input form with real exoplanet quick-load buttons, organized input cards for orbital dynamics, stellar context, physical parameters, and discovery metadata.

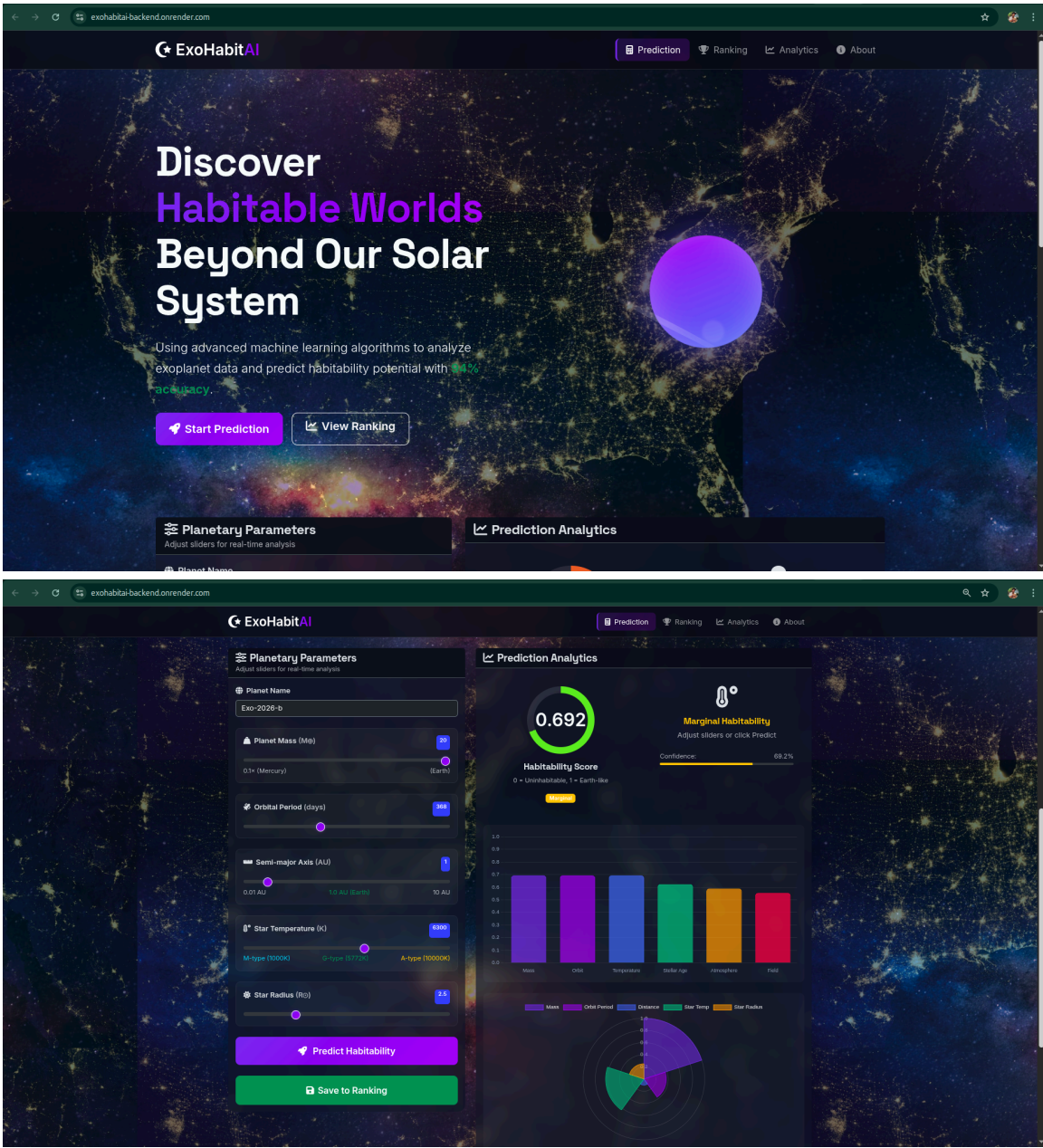
The screenshot shows a web application titled "Habitability Prediction" with the subtitle "INPUT PLANETARY PARAMETERS". At the top, there is a "LOAD REAL EXOPLANET" section with four buttons: "GJ 1132 b" (Rocky exoplanet around red dwarf), "CoRoT-2 b" (Hot Jupiter - gas giant), "GJ 1214 b" (Neptune-like mini-Neptune), and "G 9-40 b" (Super-Earth candidate). Below this are five input cards: "PLANET IDENTITY" (Designation: "e.g., Kepler-452b"), "ORBITAL DYNAMICS" (Period (days): "365.25", Semi-major Axis (AU): "1"), "STELLAR CONTEXT" (Spectral Type: "G-type", Metallicity [Fe/H]: "0"), "PHYSICAL PARAMETERS" (Mass (Earth masses): "1", Composition Type: "Rocky Planet"), and "DISCOVERY METADATA" (Surface Gravity (log g): "4.44", Discovery Year: "2024"). A "RUN ANALYSIS" button is at the bottom.

[Screenshot 4: Results Page]
Shows the prediction results with rotating 3D planet visualization, habitability score, confidence level, and action buttons including View Summary popup.

The screenshot shows the results page with a large blue circle in the center containing the text "ANALYSIS COMPLETE", "G 9-40 b", "Not Habitable", and "3.5%". Below the circle are three cards: "CATEGORY: Not Habitable", "CONFIDENCE: High", and "PRIORITY: Low". At the bottom, there is a message "Very low habitability probability - not recommended for follow-up" and three buttons: "NEW ANALYSIS", "VIEW SUMMARY", and "DASHBOARD".

[Screenshot 5: Render Deployment Dashboard]

Shows the Render service dashboard with successful deployment status, build logs confirming model and ranking data loaded, and live service URL.



8. Limitations and Future Work

Limitation	Impact	Future Work
------------	--------	-------------

Free tier cold starts on Render	First request after inactivity takes 30-60s	Upgrade to paid tier or add health-check pinger
Model based on simplified habitable zone criteria	Does not account for atmospheric composition	Incorporate atmospheric proxy features when available

9. GitHub Repositories

Repository	URL	Description
Frontend	github.com/CrownDestro/ExoHabitAI-Frontend	Next.js + React Three Fiber dashboard
Backend	github.com/CrownDestro/ExoHabitAI-Backend	Flask REST API + trained model

Conclusion

ExoHabitAI is fully deployed and operational. The frontend provides an immersive 3D orbital visualization of habitability-ranked exoplanets, while the backend API serves real-time predictions from a calibrated Logistic Regression model trained on 4,027 unique exoplanets.

The live dashboard is accessible at <https://exohabitai-frontend.vercel.app/dashboard>.